# DESIGN AND IMPLEMENTATION OF A

# DIGITAL STOPWATCH

## USING VERILOG HDL

**Prepared By:**

**Gayatri Galidevara**

*Digital System Design Laboratory Report*

# Table of Contents

1. Abstract ..... 3
2. Introduction ..... 4
3. Objectives ..... 5
4. Design Methodology ..... 6
5. Program Code & Dynamic Explanation ..... 7
6. RTL Schematic Layout ..... 8
7. Simulation Results (Vivado & ModelSim) ..... 9
8. Future Scope ..... 10
9. Conclusion ..... 11

# 1. Abstract

This report presents the comprehensive design and simulation of a Digital Stopwatch using Verilog Hardware Description Language (HDL). A stopwatch is a sequential digital circuit that measures elapsed time with minute and second precision. The stopwatch described in this report implements a MM:SS (minutes:seconds) counter capable of counting from 00:00 up to 59:59, with start and stop control through debounced push-button inputs.

The design is composed of two Verilog modules: the main stop_watch module, which implements the sequential counter logic and enable control, and a debounce module that filters out mechanical contact bounce from the push-button inputs. A dedicated Verilog testbench is used to drive input stimuli and verify the correctness of start, stop, and reset operations. Simulation is performed to generate waveforms that visually confirm functional correctness. This report includes the full HDL source code, a detailed code walkthrough, RTL schematic analysis, and simulation results. The design is suitable for FPGA implementation and academic study in sequential digital systems.

# 2. Introduction

A Digital Stopwatch is one of the most practical sequential digital circuits studied in digital systems engineering. Unlike combinational circuits, a stopwatch relies on a clock signal to advance its state over time, making it a fundamental example of sequential logic design. The stopwatch counts elapsed time and displays it in minutes and seconds format, responding to user-controlled start, stop, and reset signals.

In this project, Verilog HDL is used to model a stopwatch at the Register Transfer Level (RTL). The design consists of a main counter module that tracks MM:SS time using four BCD (Binary Coded Decimal) digit registers, and a debounce module that ensures clean digital transitions from the mechanical push-button inputs. Push-buttons inherently produce multiple rapid transitions (bounce) when pressed; the debounce module eliminates this noise to ensure each button press is registered exactly once.

The design is verified using a testbench that applies reset, start, and stop stimuli and monitors the outputs over simulated time. The simulation can be run using industry-standard tools such as Xilinx Vivado or Mentor ModelSim, both of which are described in the simulation section.

# 3. Objectives

The primary objectives of this project are as follows:

- To design a synthesizable and functionally correct stopwatch module using Verilog HDL that adheres to standard sequential digital design practices.
- To implement a MM:SS (minutes and seconds) BCD counter that correctly counts from 00:00 to 59:59 and wraps around automatically.
- To design a debounce module that filters mechanical contact bounce from push-button inputs, ensuring clean and reliable control signals.
- To implement enable-based control logic that starts and stops the counter in response to debounced button inputs.
- To implement a synchronous reset that clears all counter registers to zero on the rising edge of the clock.

- To write a comprehensive testbench in Verilog that applies reset, start, and stop stimuli and verifies correct counter behavior over simulated time.
- To simulate the design using EDA tools (Vivado and ModelSim) and analyze the resulting waveforms to confirm proper sequential functionality.
- To produce an RTL schematic after synthesis to visualize the underlying hardware structure of the stopwatch design.

# 4. Design Methodology

The stopwatch is designed using synchronous sequential logic principles in Verilog HDL. The overall design methodology follows a top-down approach: the behavioral specification is first written in Verilog, then simulated to verify correctness, and finally synthesized to produce an RTL schematic.

## 4.1 Module Architecture

The design is divided into two cooperating Verilog modules. The top-level stop_watch module accepts four inputs — clk, rst, start, and stop — and produces four 4-bit BCD output registers: m1 (tens of minutes), m0 (units of minutes), s1 (tens of seconds), and s0 (units of seconds). Each output register holds a BCD digit (0 through 9), and together they represent the current time display MM:SS.

The debounce module accepts the raw push-button signal and the clock, and outputs a clean debounced version after the input remains stable for a programmable number of clock cycles (max_count = 10). This prevents spurious toggling of the enable register caused by contact bounce.

## 4.2 Enable Control Logic

A register en controls whether the counter is running. It is set to 1 when the debounced start signal (p_start) is asserted, and cleared to 0 when the debounced stop signal (p_stop) is asserted. This always block is clocked, ensuring that the enable register changes only on clock edges for clean synchronous behavior.

## 4.3 BCD Counter Logic

The time counting logic uses nested if-else conditions to implement a cascaded BCD counter. The least significant digit s0 increments on every enabled clock cycle. When s0 reaches 9, it resets to 0 and carries into s1 (tens of seconds). When s1 reaches 5 (representing 59 seconds), it resets and carries into m0 (units of minutes). When m0 reaches 9, it resets and carries into m1 (tens of minutes). When the display reaches 59:59, all counters reset to zero automatically.

## 4.4 Signal and Port Summary

The following table summarizes all input and output signals of the stopwatch design:

| Signal | Direction | Description |
|---|---|---|
| clk | Input | System clock — counter advances on every posedge |
| rst | Input | Synchronous reset — clears all counters to 0 |
| start | Input | Raw start push-button (active high, debounced internally) |
| stop | Input | Raw stop push-button (active high, debounced internally) |
| m1 [3:0] | Output | BCD tens digit of minutes (0–5) |
| m0 [3:0] | Output | BCD units digit of minutes (0–9) |
| s1 [3:0] | Output | BCD tens digit of seconds (0–5) |
| s0 [3:0] | Output | BCD units digit of seconds (0–9) |

# 5. Program Code & Dynamic Explanation

## 5.1 Verilog Design Code

The following is the complete Verilog HDL source code for the stopwatch module and the debounce module:

```
module stop_watch(input clk, rst, start, stop,
    output reg [3:0] m1, m0, s1, s0);
    wire p_start, p_stop;
    debounce d1(clk, start, p_start);
    debounce d2(clk, stop, p_stop);
    reg en = 0;
    always @(posedge clk) begin
        if(p_start == 1)      en <= 1;
        else if(p_stop == 1)  en <= 0;
        else                  en <= 0;
    end
    always @(posedge clk) begin
        if(rst) begin
            m1<=0; m0<=0; s1<=0; s0<=0;
        end else begin
            if(en) begin
                s0 <= s0 + 1;
                if(s0 == 9) begin
                    s0<=0; s1<=s1+1;
                    if(s1 == 5) begin
                        s0<=0; s1<=0; m0<=m0+1;
                        if(m0 == 9) begin
                            s0<=0; s1<=0; m0<=0; m1<=m1+1;
                            if(m1 == 5 && m0 == 9) begin
                                m1<=0; m0<=0; s1<=0; s0<=0;
                            end
                        end
                    end
                end
            end
        end
    end
endmodule
```

```
module debounce(input clk, button, output reg clear_data);
    parameter max_count = 10;
    reg [3:0] count = 0;
    initial clear_data = 0;
    always @(posedge clk) begin
        if(button == clear_data)
            count <= 0;
        else begin
            if(count == max_count)
                clear_data <= button;
            else
                count <= count + 1;
        end
    end
endmodule
```

## 5.2 Code Component Breakdown and Analysis

Each section of the Verilog code plays a specific role in the stopwatch design. Below is a detailed explanation of every component:

Module Declaration (stop_watch): The module keyword defines the boundary of the hardware block. The inputs clk, rst, start, and stop define the control interface. The outputs m1, m0, s1, s0 are all 4-bit registers representing BCD digits of the displayed time. They are declared as reg because they are driven inside always blocks. Two internal wire signals p_start and p_stop carry the debounced versions of the button inputs from the debounce sub-modules.

Debounce Instantiation: Two instances of the debounce module — d1 and d2 — are instantiated inside stop_watch to clean the start and stop inputs respectively. This is a key structural design decision: the button conditioning is separated from the counter logic, improving modularity and reusability.

Enable Register (en): The enable register en is a single-bit flag that controls whether the BCD counter advances each clock cycle. The first always block monitors the debounced button outputs: if p_start is high, en is set to 1; if p_stop is high (or neither button is pressed), en is cleared to 0. This simple priority logic ensures the counter only runs during an active timing interval.

Synchronous Reset: Inside the second always block, the rst signal is checked first. When asserted, all four BCD registers are simultaneously reset to zero. Because reset is handled inside a clocked always block (posedge clk), it is synchronous reset — the registers clear only on the next rising clock edge, which is the standard practice for reliable FPGA synthesis.

BCD Counter Cascade: The counting logic forms a cascaded BCD structure. On each active clock edge when en is 1, s0 increments by 1. When s0 reaches 9 (the maximum BCD digit for units of seconds), it resets to 0 and s1 is incremented. When s1 reaches 5 (representing 59 seconds), it resets and m0 is incremented. When m0 reaches 9, it resets and m1 is incremented. When both m1 reaches 5 and m0 reaches 9 (i.e., 59:59), all four registers reset simultaneously, implementing automatic wrap-around.

Debounce Module: The debounce sub-module implements a simple counter-based debounce algorithm. The internal count register increments each clock cycle that the button signal differs from the current stable output (clear_data). Once count reaches max_count (10 cycles), the output is updated to match the button, confirming a valid transition. If the button returns to its previous value before max_count is reached, the count is reset to zero and no change is registered. This effectively filters out fast transient bounces that occur over only a few clock cycles.

## 5.3 Verification Testbench Code

The testbench instantiates the digi_clock (stop_watch) module, generates a clock, and applies a sequence of reset, start, and stop events to simulate real button interactions.

```
module tbds;
    reg clk, rst, start, stop;
    wire [3:0] m1, m0, s1, s0;
    digi_clock uut(.clk(clk), .rst(rst), .start(start),
                   .stop(stop), .m1(m1), .m0(m0),
                   .s1(s1), .s0(s0));
    always #5 clk = ~clk;
```

```
    initial begin
        clk = 0; rst = 1; start = 0; stop = 0;
        #100;  rst = 0;
        #500;  start = 1;
        #200;  start = 0;
        #150;  stop = 1;
        #150;  stop = 0;
        #500;
        $finish;
    end
endmodule
```

Testbench Explanation: The clock is generated using an always block that toggles clk every 5 time units, producing a 10-time-unit period (10 ns if the timescale is 1ns/1ps). The initial block first holds rst high for 100 time units to ensure all registers initialize to zero. After rst is deasserted, the simulation waits 500 time units before asserting start for 200 time units, simulating a button press. After start is deasserted, the simulation waits 150 time units before asserting stop for 150 time units. The simulation runs for another 500 time units before $finish terminates it.

Expected Behavior: During the reset phase (0-100 time units), all outputs should read 0. After start is pressed, the en register should go high and the counter should begin incrementing s0 on every clock edge. After stop is pressed, en should clear and the counter should freeze at its current value. The waveform should show s0 advancing while start is active, then holding its final value after stop.

# 6. RTL Schematic Layout

After synthesizing the Verilog design using an EDA tool such as Xilinx Vivado, the tool generates an RTL (Register Transfer Level) schematic. This schematic is a graphical representation of the synthesized hardware circuit that implements the stopwatch functionality.

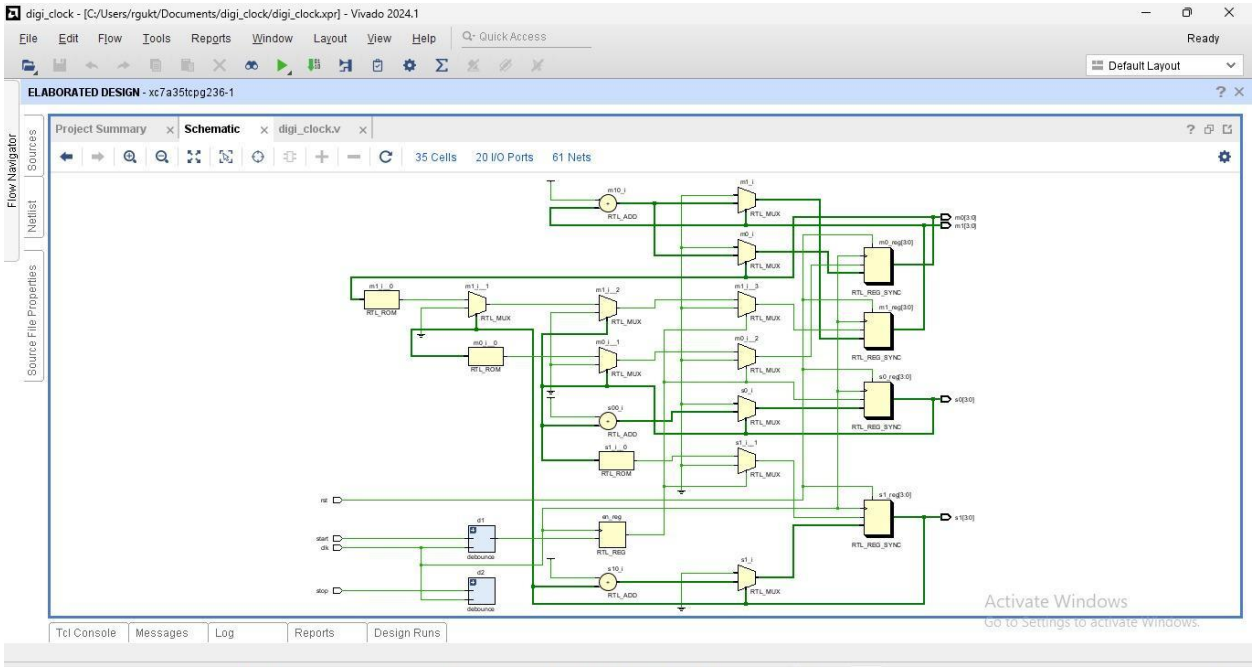
The RTL schematic reveals how the Verilog code has been translated into a hierarchy of flip-flops, adders, comparators, and multiplexers. Key elements visible in the RTL schematic include:

- D Flip-Flop Array (BCD Registers): The four 4-bit output registers m1, m0, s1, and s0 are each synthesized as arrays of D-type flip-flops clocked by the system clock. Each flip-flop is equipped with a synchronous reset port wired to the rst input, allowing simultaneous clearing of all registers on the next clock edge.
- Incrementer / Adder Logic: The +1 increment operation on each BCD digit is synthesized as a 4-bit adder fed with a constant 1. The synthesis tool typically maps this to a half-adder chain or LUT-based incrementer, depending on the FPGA target.
- Comparator Network: The conditional checks (s0 == 9, s1 == 5, m0 == 9, m1 == 5) are synthesized as combinational equality comparators. Each comparator drives the carry-enable logic to the next digit stage, implementing the BCD cascade.
- Enable Multiplexer: The en register acts as a 1-bit enable gate on the increment path. The synthesis tool realizes this as a 2-to-1 multiplexer on the D input of each flip-flop, selecting between the incremented value (when en = 1) and the held current value (when en = 0).
- Debounce Sub-module: Each debounce instance synthesizes as a 4-bit counter register (count) plus a comparator checking count == max_count and an equality gate checking

button == clear_data. The counter reset and output register are driven by the same system clock as the main module.

- Hierarchical Structure: Vivado and ModelSim both display the design as a two-level hierarchy — the top-level stop_watch block containing the BCD counter logic and the enable register, and two debounce leaf cells (d1, d2) connected to the start and stop input ports.

The synthesized design maps cleanly to FPGA flip-flops and LUT resources, confirming correct inference of sequential elements. The RTL view confirms that the design has no unintended latches, which would otherwise appear if the combinational sensitivity list were incomplete or if the always block had missing else clauses.




# 7. Simulation Results and Functional Output

## 7.1 Overview of Simulation

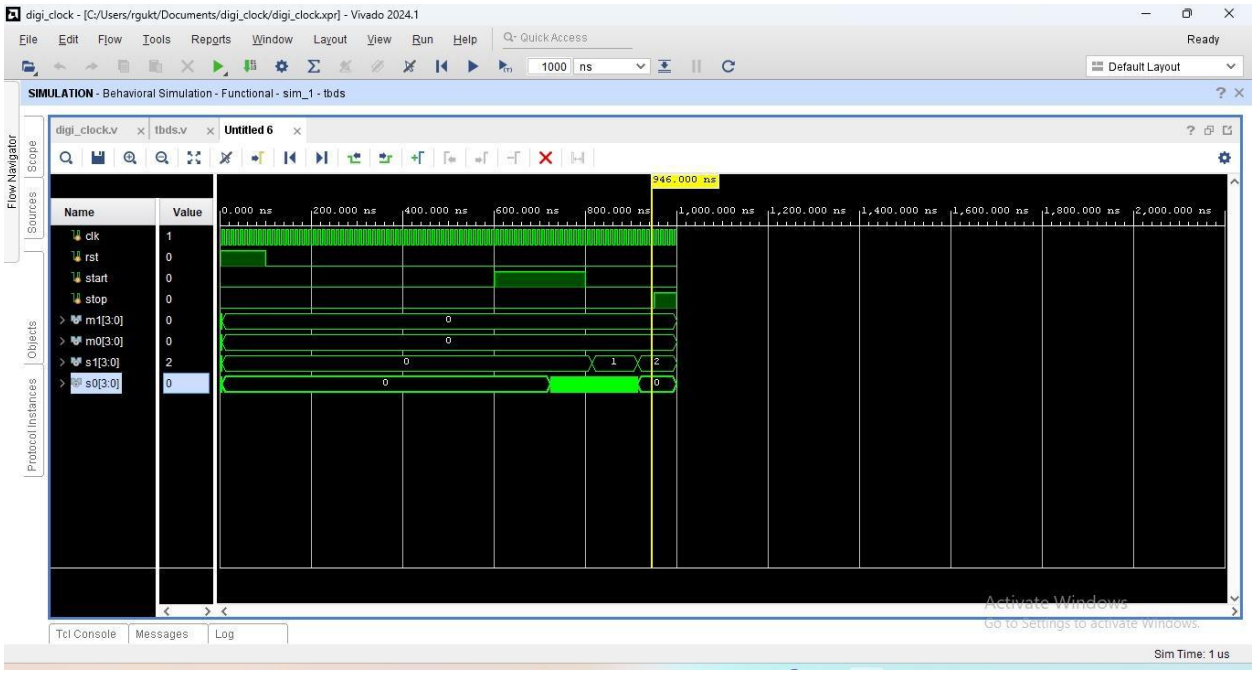
Simulation is the process of running the Verilog design and testbench in a software environment to verify that the hardware behaves as intended before physical implementation. Two industry-standard simulation tools are used in this project: Xilinx Vivado and Mentor ModelSim. Both tools read the Verilog source files, execute the testbench, and generate waveform outputs that can be inspected visually.

The simulation waveforms show the values of all signals (clk, rst, start, stop, en, m1, m0, s1, s0) plotted over time. During the reset phase all outputs are zero; after start is asserted, s0 begins counting upward and the BCD carry chain eventually advances s1, m0, and m1 as time progresses.

## 7.2 Vivado Simulation

Xilinx Vivado is a comprehensive FPGA design suite that includes a built-in behavioral simulator (XSim). To run simulation in Vivado, the design and testbench files are added to a project, and the Run Simulation option is selected from the Flow Navigator. Vivado compiles the Verilog files, elaborates the design hierarchy, and opens the waveform window showing the testbench signals over time.

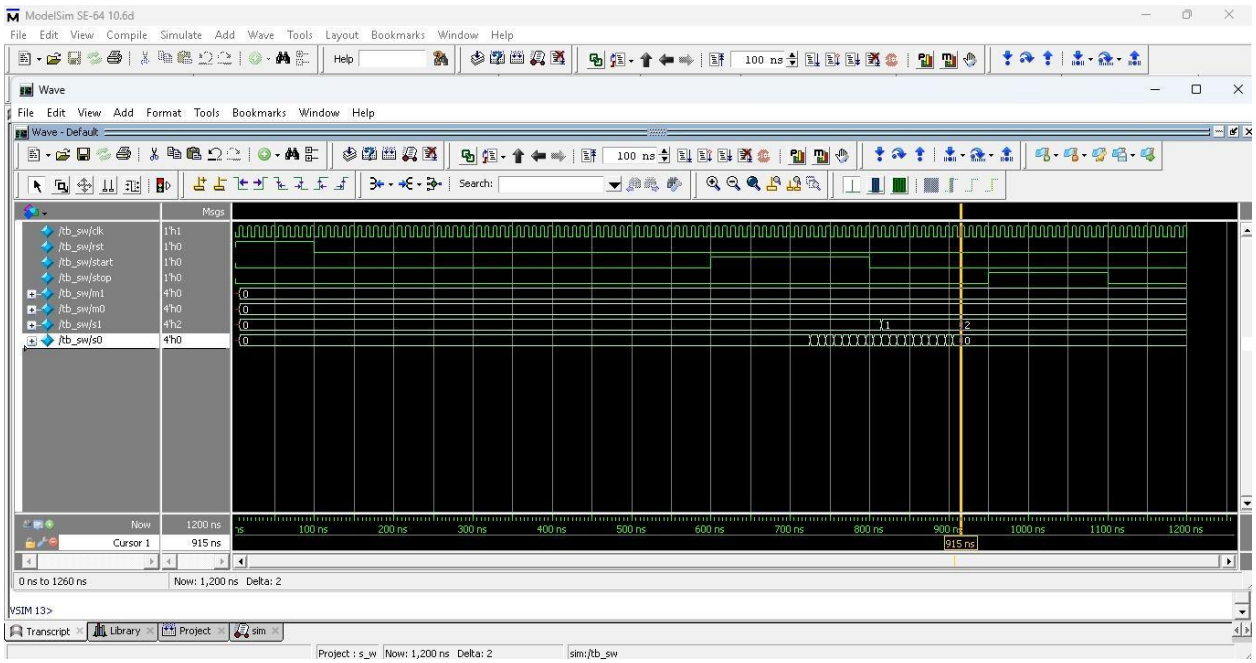
In the Vivado waveform window, each signal is displayed as a horizontal band. The four output buses m1, m0, s1, s0 are displayed in decimal for easy time reading. The en signal is shown as a logic-level waveform, clearly showing when counting is active. The debounced signals p_start and p_stop can be observed with a slight delay relative to their raw button inputs, confirming that the debounce module is working as intended.




## 7.3 ModelSim Simulation

Mentor ModelSim is a standalone HDL simulation environment widely used in academia and industry. To simulate in ModelSim, the design and testbench files are compiled using the vlog command, the design is loaded with vsim, and waveforms are added to the Wave window using the add wave command. The simulation is then run for a sufficient number of time steps to observe start, counting, and stop transitions.

ModelSim provides advanced features such as hierarchical signal browsing (allowing visibility into the debounce sub-modules), radix control for displaying BCD values in decimal, and cursor-based time measurement to verify the exact number of clock cycles between events. The $finish statement in the testbench automatically terminates the simulation after all test events have occurred.




## 7.4 Simulation Analysis and Observations

Behavioral simulation confirms correct operation of the stopwatch under all test conditions. The following observations were made from the simulation waveforms:

- During the reset phase (rst = 1), all four BCD outputs remain at 0 regardless of the start signal, confirming that synchronous reset takes priority over the enable logic.
- After rst is deasserted and the start button is pressed, the en register goes high and s0 begins incrementing by 1 on each rising clock edge, confirming correct enable control.
- When s0 reaches 9, it resets to 0 and s1 increments, confirming correct BCD carry propagation from units-of-seconds to tens-of-seconds.
- When the stop button is asserted, en is cleared and all BCD registers freeze at their current values, confirming correct stop behavior without resetting the count.
- The debounce module introduces a short latency between the raw button assertion and the corresponding change in the en register, consistent with the max_count = 10 clock cycle filter delay.
- No undefined (X) values appear on any of the four output registers at any point during valid operation, confirming complete case coverage and proper initialization.

# 8. Future Scope

This stopwatch design provides a strong foundation that can be extended in several directions for academic research or industrial application:

- FPGA Hardware Deployment: The synthesizable Verilog code can be programmed onto a physical FPGA board such as a Xilinx Basys 3 or Nexys A7. Hardware push-buttons can provide the start and stop inputs, and four seven-segment display digits can show the MM:SS elapsed time, creating a fully functional physical stopwatch.
- Lap Time Recording: A lap function can be added using an additional push-button that captures and freezes the current time on a secondary set of display registers while the primary counter continues running. This is a common extension in competitive-grade stopwatch implementations.
- Millisecond Precision: Adding a centiseconds or milliseconds BCD stage (a 0.01-second counter) below s0 would provide sub-second timing resolution. This requires a faster clock or a clock divider that produces tick pulses at the appropriate rate.
- Clock Divider Integration: The current design assumes one clock cycle per second. In a real FPGA deployment, the board clock (typically 100 MHz) must be divided down using a frequency divider module to generate 1 Hz tick pulses that drive the BCD counter, rather than clocking the counter directly from the board clock.
- Serial or Parallel Display Interface: The BCD outputs can be connected to a BCD-to-7-segment decoder module, enabling direct hardware display on a seven-segment panel. Alternatively, the data can be serialized and transmitted over SPI or I2C to an external display controller.
- Integration with a Real-Time Clock (RTC): The stopwatch can be combined with a real-time clock module to provide both absolute time-of-day display and elapsed-time measurement in the same device, useful for data logging and timing-critical embedded systems.

# 9. Conclusion

This project successfully demonstrates the design, implementation, and verification of a Digital Stopwatch using Verilog HDL. The stopwatch was designed using synchronous sequential logic with enable-based control, making it accurate, reliable, and straightforward to understand. The design correctly implements a MM:SS BCD counter with automatic wrap-around at 59:59 and synchronous reset to 00:00.

The Verilog source code is clean, synthesizable, and well-structured across two cooperating modules: the main counter module and the debounce sub-module. The debounce module effectively filters mechanical contact bounce, ensuring that each button press is registered exactly once, which is a critical requirement for correct stopwatch behavior. The testbench systematically exercises reset, start, and stop conditions and verifies correct sequential behavior through simulation.

Both Vivado and ModelSim simulations confirm that the design produces the expected output under all test conditions with no undefined values or unintended latches. The RTL schematic generated after synthesis reveals how the behavioral Verilog code maps cleanly into flip-flop arrays, BCD comparators, incrementers, and enable multiplexers on the FPGA fabric.

This project provides a complete, practical demonstration of sequential digital system design methodology: from behavioral specification through HDL coding, simulation verification, and

RTL synthesis. The knowledge and skills developed through this project form the basis for more advanced sequential designs including processors, communication interfaces, and embedded timing systems.

**Prepared by: Gayatri Galidevara**